

Observatoire Intelligent des Prix Immobiliers

Projet DataOps & MLOps

De l'ingestion des données au déploiement et au monitoring d'un service de prédiction immobilière.

Groupe: Mlops-MEIA-v2:

Nohaila Ichou • Moussaoui Abdelkarim • Hajar El Khalidi • Zainab Rhouni
Douaa Bounadar • Salama Boussaid • Malika Chaaban • Youness Zouhairi

Encadrant : Pr. Mohammed Ait Daoud | Co-encadrant: Taoufiq Amzil
Année universitaire 2025–2026



Objectif du projet

Construire une chaîne DataOps / MLOps complète et reproductible autour de la prédiction des prix immobiliers.



Logique générale : Collecter → Fiabiliser → Apprendre → Industrialiser

Organisation du projet en 4 sprints

Sprint 1

Cadrage, Data Strategy
et Ingestion

22 SP

Sprint 2

Transformation, Qualité
et Orchestration

24 SP

Sprint 3

Machine Learning
et MLflow

24 SP

Sprint 4

FastAPI, Docker, CI/CD,
Déploiement et Monitoring

25 SP

95 / 95

Story Points réalisés

24

User Stories

96

Subtasks nominatives

100 %

Taux de réalisation

Équipe et responsabilités

01

Nohaila Ichou**Product Owner**

Vision produit · backlog · priorisation

02

Moussaoui Abdelkarim**Scrum Master**

Sprints · coordination · obstacles

03

Hajar El Khalidi**Data Engineer 1**

Collecte · ingestion · orchestration

04

Zainab Rhouni**Data Engineer 2**

Stockage · transformation · structuration

05

Douaa Bounadar**Data Quality Engineer**

Règles · validation · tests

06

Salama Boussaid**ML Engineer 1**

Feature engineering · entraînement

07

Malika Chaaban**ML Engineer 2**

Expérimentation · MLflow · registry

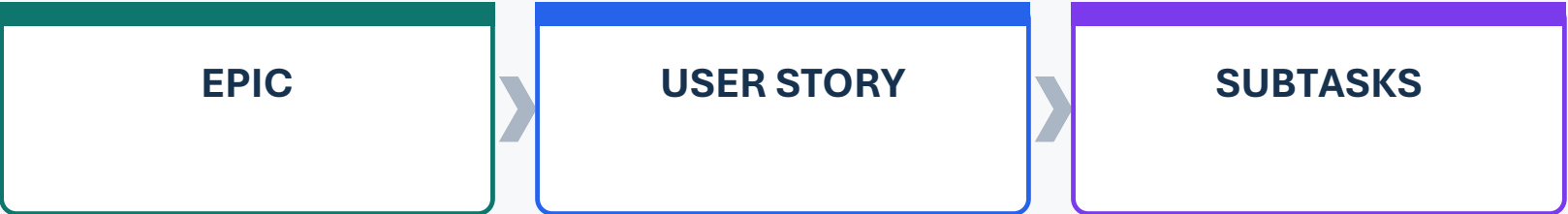
08

Youness Zouhairi**DevOps / Deployment**

API · Docker · CI/CD · monitoring

Jira : backlog et traçabilité

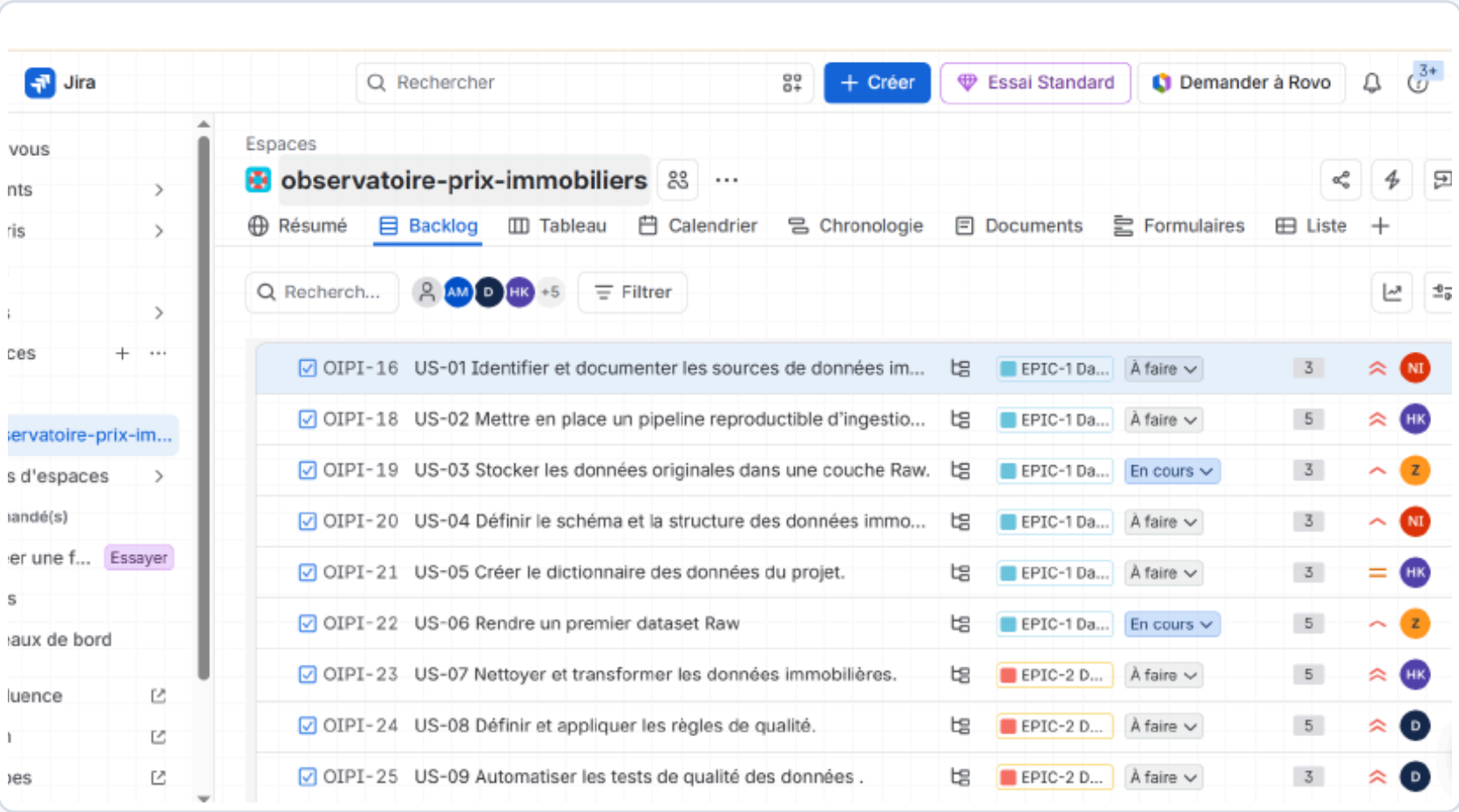
Hiérarchie utilisée :



Les activités de pilotage du Scrum Master sont suivies séparément sous forme de Tasks Jira.



Avancement des Epics dans Jira



Vue du Product Backlog

4 Epics · 24 User Stories · 96 Subtasks · 24 Tasks Scrum Master

Suivi Agile : résultat des 4 sprints

6 / 6

Stories terminées – Sprint 1

6 / 6

Stories terminées – Sprint 2

6 / 6

Stories terminées – Sprint 3

6 / 6

Stories terminées – Sprint 4

0

Story reportée

Cadrage, Data Strategy, organisation Agile et ingestion.									
☐	☒	OIPI-16	US-01 Identifier et documenter les sources de données immo...	EPIC-1 Da...	À faire	3	NI		
☐	☒	OIPI-19	US-03 Stocker les données originales dans une couche Raw.	EPIC-1 Da...	À valider	3	Z		
☐	☒	OIPI-18	US-02 Mettre en place un pipeline reproductible d'ingestion d...	EPIC-1 Da...	À valider	5	HK		
☐	☒	OIPI-20	US-04 Définir le schéma et la structure des données immobili...	EPIC-1 Da...	À faire	3	NI		
☐	☒	OIPI-21	US-05 Créer le dictionnaire des données du projet.	EPIC-1 Da...	À faire	3	HK		
☐	☒	OIPI-22	US-06 Rendre un premier dataset Raw	EPIC-1 Da...	En cours	5	Z		
☐	☒	OIPI-137	SM-01 Configurer le projet Scrum dans Jira		À faire	-	AM		
☐	☒	OIPI-138	SM-02 Créer et organiser les quatre Epics.		À faire	-	AM		
☒	☒	OIPI-139	SM-03 Créer et structurer le Product Backlog		À faire	-	AM		
☐	☒	OIPI-140	SM-04 Créer les quatre sprints dans Jira.		À faire	-	AM		

Exemple de board Jira pendant le Sprint 1

95 / 95 Story Points réalisés

100 % de la charge planifiée a été terminée sur les 4 sprints.

24 User Stories • 96 Subtasks

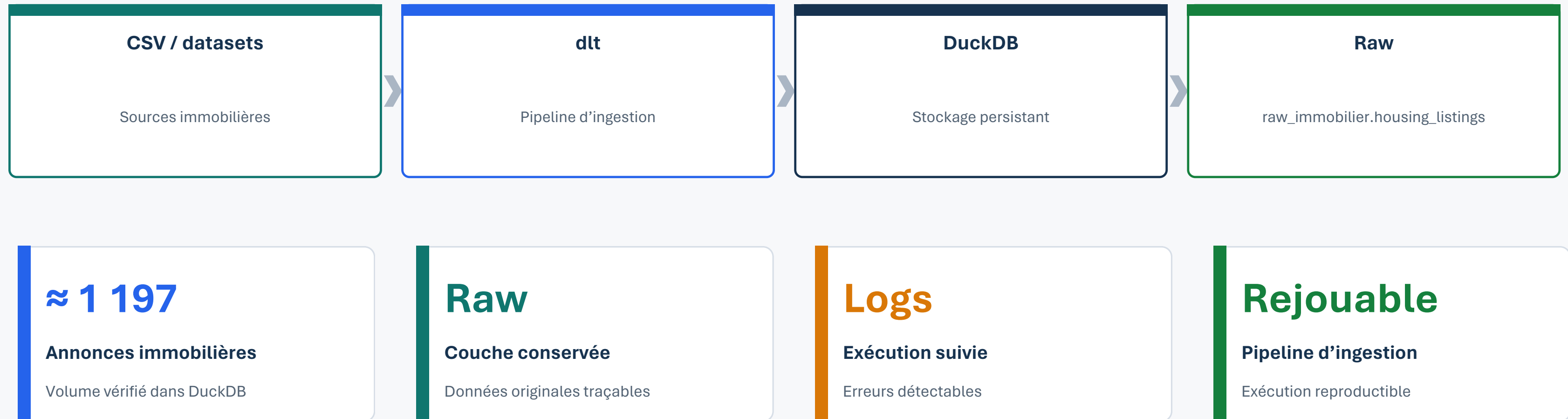
toutes les User Stories prévues ont été finalisées, sans report.

24 Tasks Scrum Master

Sprint Planning, suivi Jira, coordination, Reviews et Retrospectives.

Ingestion des données avec dlt

Premier maillon de la chaîne DataOps : récupérer les données et les rendre reproductibles.



Le dataset Raw devient l'entrée de la transformation dbt et du contrôle de qualité du Sprint 2.

Transformation & structuration des données (dbt)

Contexte

Je prends les données brutes ingérées dans DuckDB et je les transforme avec dbt (SQL) en couches successives.

Nettoyage (Staging)

Modèle stg_housing_listings : suppression des prix et surfaces invalides (≤ 0), recalcul du prix au m² pour garantir sa cohérence, ajout d'un timestamp de fraîcheur.

Agrégation (Intermediate)

Modèles int_housing_by_city et int_housing_by_type : calcul des moyennes, min, max de prix par ville et par type de bien.

Tables finales (Marts)

3 tables prêtes à l'emploi :
mart_price_by_city,
mart_price_per_sqm_by_city,
mart_price_by_type.

DATA PIPELINE FLOW

1. RAW LAYER

raw_immobilier.housing_listings DuckDB



2. STAGING LAYER

stg_housing_listings



3. INTERMEDIATE LAYER

int_housing_by_city

int_housing_by_type



4. DATA MARTS

mart_price_by_city

mart_price_per_sqm_by_city

mart_price_by_type

Sécuriser stg_housing_listings : mes 17 tests dbt

17 tests structurels (fichier schema.yml)

unique

garantit l'unicité de listing_id, ainsi que des clés d'agrégation city et property_type dans les tables intermédiaires.

accepted_values

contrôle que property_type ne contient que des valeurs autorisées (Appartement, Villa, Maison, etc.).

not_null

vérifie l'absence de valeurs nulles sur les colonnes critiques : listing_id, price_mad, surface_m2, city, property_type

Ces 17 tests couvrent l'intégralité de mes modèles dbt. La Data Quality Engineer ajoute ensuite des tests métier supplémentaires

```
11:09:03 2 of 17 START test not_null_int_housing_by_city_city ..... [RUN]
11:09:03 2 of 17 PASS not_null_int_housing_by_city_city ..... [PASS in 0.05s]
11:09:03 3 of 17 START test not_null_int_housing_by_type_property_type ..... [RUN]
11:09:03 3 of 17 PASS not_null_int_housing_by_type_property_type ..... [PASS in 0.04s]
11:09:03 4 of 17 START test not_null_mart_price_by_city_city ..... [RUN]
11:09:03 4 of 17 PASS not_null_mart_price_by_city_city ..... [PASS in 0.04s]
11:09:03 5 of 17 START test not_null_mart_price_by_type_property_type ..... [RUN]
11:09:03 5 of 17 PASS not_null_mart_price_by_type_property_type ..... [PASS in 0.04s]
11:09:03 6 of 17 START test not_null_mart_price_per_sqm_by_city_city ..... [RUN]
11:09:03 6 of 17 PASS not_null_mart_price_per_sqm_by_city_city ..... [PASS in 0.04s]
11:09:03 7 of 17 START test not_null_stg_housing_listings_city ..... [RUN]
11:09:03 7 of 17 PASS not_null_stg_housing_listings_city ..... [PASS in 0.04s]
11:09:03 8 of 17 START test not_null_stg_housing_listings_listing_id ..... [RUN]
11:09:03 8 of 17 PASS not_null_stg_housing_listings_listing_id ..... [PASS in 0.04s]
11:09:03 9 of 17 START test not_null_stg_housing_listings_price_mad ..... [RUN]
11:09:03 9 of 17 PASS not_null_stg_housing_listings_price_mad ..... [PASS in 0.04s]
11:09:03 10 of 17 START test not_null_stg_housing_listings_property_type ..... [RUN]
11:09:03 10 of 17 PASS not_null_stg_housing_listings_property_type ..... [PASS in 0.04s]
11:09:03 11 of 17 START test not_null_stg_housing_listings_surface_m2 ..... [RUN]
11:09:03 11 of 17 PASS not_null_stg_housing_listings_surface_m2 ..... [PASS in 0.04s]
11:09:03 12 of 17 START test unique_int_housing_by_city_city ..... [RUN]
11:09:03 12 of 17 PASS unique_int_housing_by_city_city ..... [PASS in 0.05s]
11:09:03 13 of 17 START test unique_int_housing_by_type_property_type ..... [RUN]
11:09:03 13 of 17 PASS unique_int_housing_by_type_property_type ..... [PASS in 0.07s]
11:09:04 14 of 17 START test unique_mart_price_by_city_city ..... [RUN]
11:09:04 14 of 17 PASS unique_mart_price_by_city_city ..... [PASS in 0.10s]
11:09:04 15 of 17 START test unique_mart_price_by_type_property_type ..... [RUN]
11:09:04 15 of 17 PASS unique_mart_price_by_type_property_type ..... [PASS in 0.05s]
11:09:04 16 of 17 START test unique_mart_price_per_sqm_by_city_city ..... [RUN]
11:09:04 16 of 17 PASS unique_mart_price_per_sqm_by_city_city ..... [PASS in 0.04s]
11:09:04 17 of 17 START test unique_stg_housing_listings_listing_id ..... [RUN]
11:09:04 17 of 17 PASS unique_stg_housing_listings_listing_id ..... [PASS in 0.05s]
11:09:04 Finished running 17 data tests in 0 hours 0 minutes and 2.80 seconds (2.80s).
11:09:04 Completed successfully
11:09:04 Done. PASS=17 WARN=0 ERROR=0 SKIP=0 NO-OP=0 REUSED=0 TOTAL=17
```

Data Quality = Vérification du respect de ces règles.

Vérification automatique du respect des règles

Est-ce que les données sont complètes ?

Est-ce que les valeurs sont valides ?

Est-ce que les données sont cohérentes ?

Est-ce que les règles métier sont respectées ?

Data Contract = les règles

Un Data Contract, c'est finalement un contrat entre les différentes parties du pipeline de données.

quelles colonnes doivent exister ?

quelles valeurs sont acceptées ?

quelles contraintes doivent être respectées ?

quelles règles métier doivent être vérifiées ?

le prix en dirhams et ne peut pas être vide
et sa valeur doit être supérieure à 0

YAML = le contrat + les contrôles déclarés

description → documentation

not_null → Un test dbt

name → le nom de la colonne.



```
11      not_null
12      - name: price_mad
13        description: "Prix en dirhams (valeurs > 0)"
14        tests:
15          - not_null
16      - name: surface_m2
17        description: "Surface en m² (valeurs > 0)"
18        tests:
19          - not_null
```

Objectif : éviter le « Garbage In, Garbage Out » et certifier les données consommées par les ML Engineers.

Un test métier SQL est une règle de vérification qui permet à la Data Quality de contrôler automatiquement qu'une donnée respecte les exigences définies dans le Data Contract.

Exemples:

Test métier : cohérence du prix au m²

SQL

```
SELECT
  listing_id,
  price_mad,
  surface_m2,
  price_per_sqm_calculated
FROM {{ ref('stg_housing_listings') }}
WHERE surface_m2 > 0
  AND ABS((price_mad / surface_m2) - price_per_sqm_calculated) > 2
```

Test métier : surface logique

SQL

```
SELECT
  listing_id,
  property_type,
  surface_m2
FROM {{ ref('stg_housing_listings') }}
WHERE surface_m2 <= 0
  OR (property_type = 'Appartement' AND surface_m2 < 9)
```

Objectif : éviter le « Garbage In, Garbage Out » et certifier les données consommées par les ML Engineers.

Documentation et Data Lineage

le parcours de nos données dans le pipeline.



Graphe de dépendances généré par dbt

Source

raw_immobilier.housing_listings

Staging

le modèle dbt qui contient les données immobilières après une première préparation/nettoyage.

Tests métier

contrôle stg_housing_listings

Intermediate & Marts

Intermediate = transformations intermédiaires
Marts = résultats finaux prêts à la consommation

Préparation des données pour le Machine Learning (feature engineering)

1 197

Annonces

19

Colonnes initiales

109 → 49

Catégories de localisation

80 / 20

Train / Test

Nettoyage

- Suppression des lignes sans prix et des doublons
- Filtrage des prix < 50 000 ou > 30 000 000 MAD
- Imputation médiane des 25 valeurs manquantes de rooms

Prévention des biais

- Exclusion de address, elevator, terrace, parking (≈98 % manquantes)
- Exclusion de price_per_sqm_calculated pour éviter la fuite de données

Features retenues

Numériques : surface_m2, bedrooms, bathrooms, floor, rooms
Catégorielles : city, property_type, localisation_grouped
Cible : price_mad

Comparaison des modèles — expérimentation initiale

R^2 sur le jeu de test

XGBoost



0.900

RMSE 1 539 903 MAD · MAE 825 580 MAD

**Linear
Regression**



0.885

RMSE 1 645 188 MAD · MAE 902 793 MAD

Random Forest



0.873

RMSE 1 734 450 MAD · MAE 912 237 MAD

3 modèles comparés

- Linear Regression (baseline)
- Random Forest Regressor — 200 arbres
- XGBoost Regressor — 300 estimateurs, learning_rate = 0.05

Décision de production

Random Forest est ensuite retenu pour son compromis performance / robustesse et sa stabilité face aux valeurs extrêmes.

Les métriques de production sont ensuite retracées systématiquement dans MLflow.

MLflow : tracking et Model Registry

Le pipeline de modélisation (préparation des données, entraînement et comparaison de trois modèles de régression) est complété par une couche de suivi, de versionnement et de surveillance.

Experiment tracking

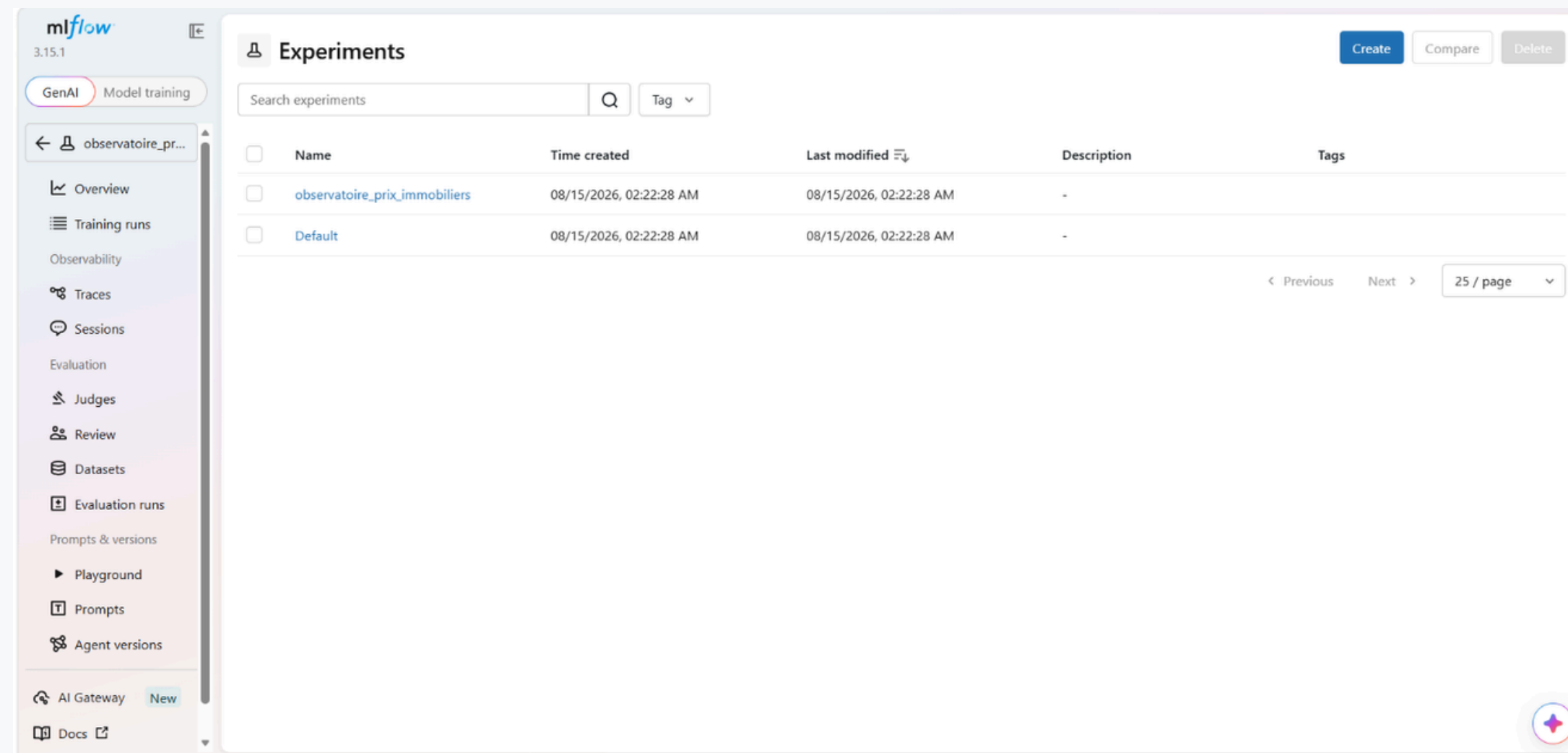
Tracer chaque expérimentation de manière reproductible avec MLflow : paramètres, métriques et artefacts.

Model Registry

Enregistrer et versionner le modèle retenu, et gérer son cycle de vie par stages.

Détection de dérive

Comparer les données de production à une baseline statistique des features d'entraînement.



MLflow : tracking et Model Registry

Les métriques obtenues pour les trois modèles, telles que loguées dans MLflow :

observatoire_prix_immobiliers > Runs >

linear_regression

OverviewModel metricsSystem metricsTracesArtifacts

Description

No description

Metrics (3)

Q Search metrics

Metric	Value	Models
rmse	1571716.5632468017	model
mae	849020.497876235	model
r2	0.902946278089175	model

observatoire_prix_immobiliers > Runs >

random_forest

OverviewModel metricsSystem metricsTracesArtifacts

Description

No description

Metrics (3)

Q Search metrics

Metric	Value	Models
rmse	1612481.7669745607	model
mae	874210.2138984264	model
r2	0.8978464743585381	model

observatoire_prix_immobiliers > Runs >

xgboost

OverviewModel metricsSystem metricsTracesArtifacts

Description

No description

Metrics (3)

Q Search metrics

Metric	Value	Models
rmse	1932469.4658429443	model
mae	962490.5267737393	model
r2	0.8532801021313856	model

Modèle	RMSE	MAE	R²
Linear Regression	1 571 717	849 020	0,903
Random Forest	1 612 482	874 210	0,898
XGBoost	1 932 469	962 491	0,853

mlflow3.15.1

GenAIModel training

← observatoire_prix_i...

RunsModelsTracesModel registry

Registered Models >prix_immobilier_maroc

Created Time: 08/15/2026, 02:35:56 AMLast Modified: 08/15/2026, 02:56:05 PM

> Description [Edit](#)

> Tags

▼ Versions [Compare](#)

Version	Registered at ⌵	Created by	Tags	Aliases	Description
✔ Version 3	08/15/2026, 02:56:04 PM		Add	Add	Modele linear_regression rete...
✔ Version 2	08/15/2026, 03:45:00 AM		Add	Add	Modele linear_regression rete...
✔ Version 1	08/15/2026, 02:35:56 AM		Add	Add	Modele linear_regression rete...

< PreviousNext > 25 / page ⌵

New model registry UI

Monitoring ML : détection de dérive par PSI

Baseline

Les statistiques des features d'entraînement sont sauvegardées dans feature_baseline.json.

Comparaison

Un nouveau lot de données est comparé à la baseline avec le Population Stability Index (PSI).

$$PSI = \sum_{i=1}^n (Actual\%_i - Expected\%_i) \times \ln \left(\frac{Actual\%_i}{Expected\%_i} \right)$$

Seuils PSI utilisés

PSI < 0.10

Pas de dérive significative

0.10 ≤ PSI < 0.25

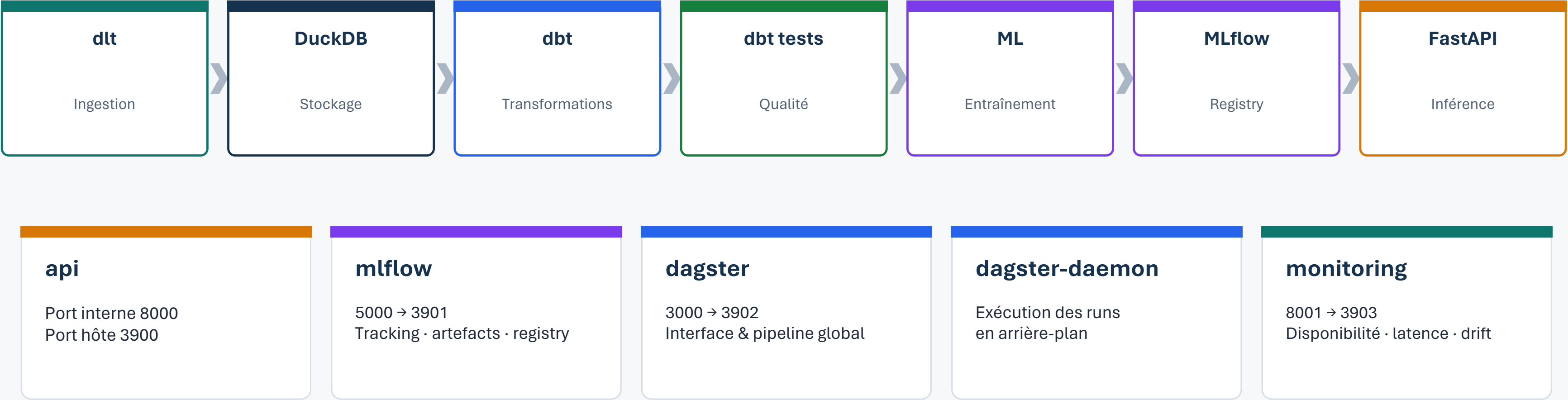
Dérive modérée — à surveiller

PSI ≥ 0.25

Dérive importante — réentraînement à envisager

```
(venv) PS C:\Users\hp\Desktop\projet mlops\mlops_meia_v2> python ml/monitoring/drift_check.py --new-data data/stg_export.csv
feature      type      psi status
surface_m2   numeric  0.023531 OK
bedrooms     numeric  0.006012 OK
bathrooms    numeric  0.003703 OK
floor        numeric  0.003952 OK
rooms        numeric  0.004257 OK
city         categorical 0.001982 OK
property_type categorical 0.002047 OK

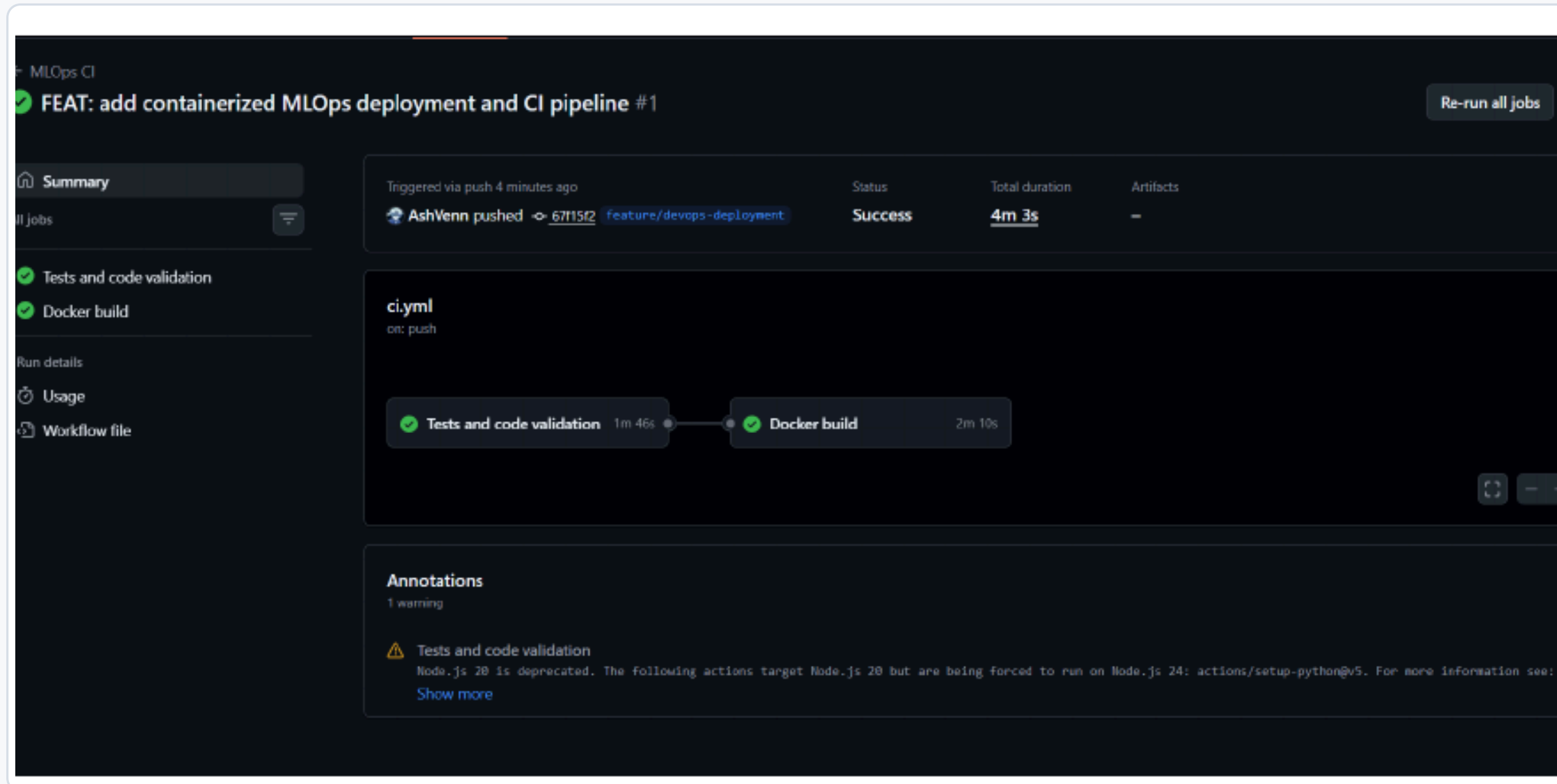
✅ Pas de derive significative detectee.
```



Volumes persistants : `housing_data` · `mlflow_data` · `mlflow_artifacts` · `dagster_data` · `monitoring_data`

La persistance évite la perte de DuckDB, des artefacts MLflow, de l'état Dagster et de la baseline de monitoring.

Intégration continue avec GitHub Actions



Pipeline CI exécutée avec succès

1. Dépendances

Installation de l'environnement

2. Qualité du code

Ruff + pytest

3. Validation Data

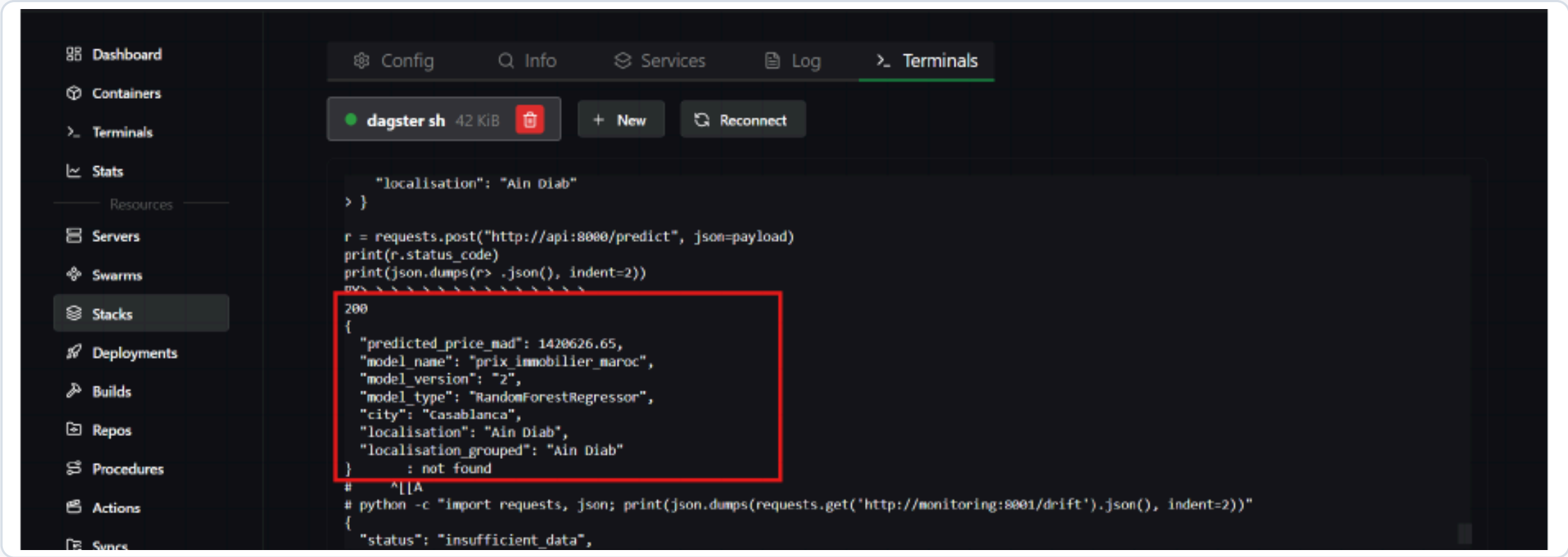
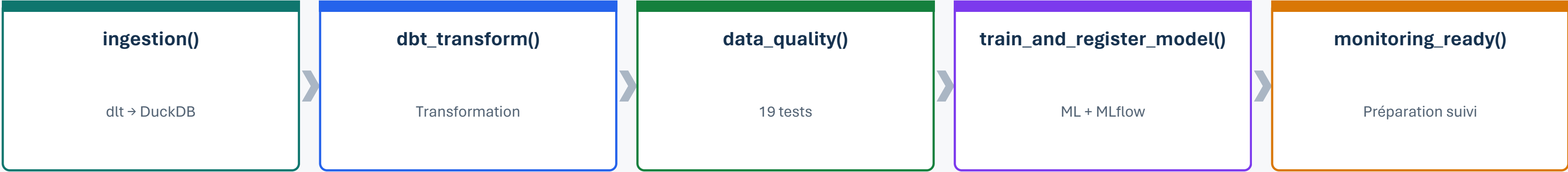
dbt parse / contrôles

4. Build

Construction de l'image Docker

Résultat final : CI verte avant passage au déploiement.

Orchestration Dagster de bout en bout



DAG global

Ingestion → transformation → qualité →
entraînement → registry → monitoring

Ingestion → transformation → qualité → entraînement → registry → monitoring

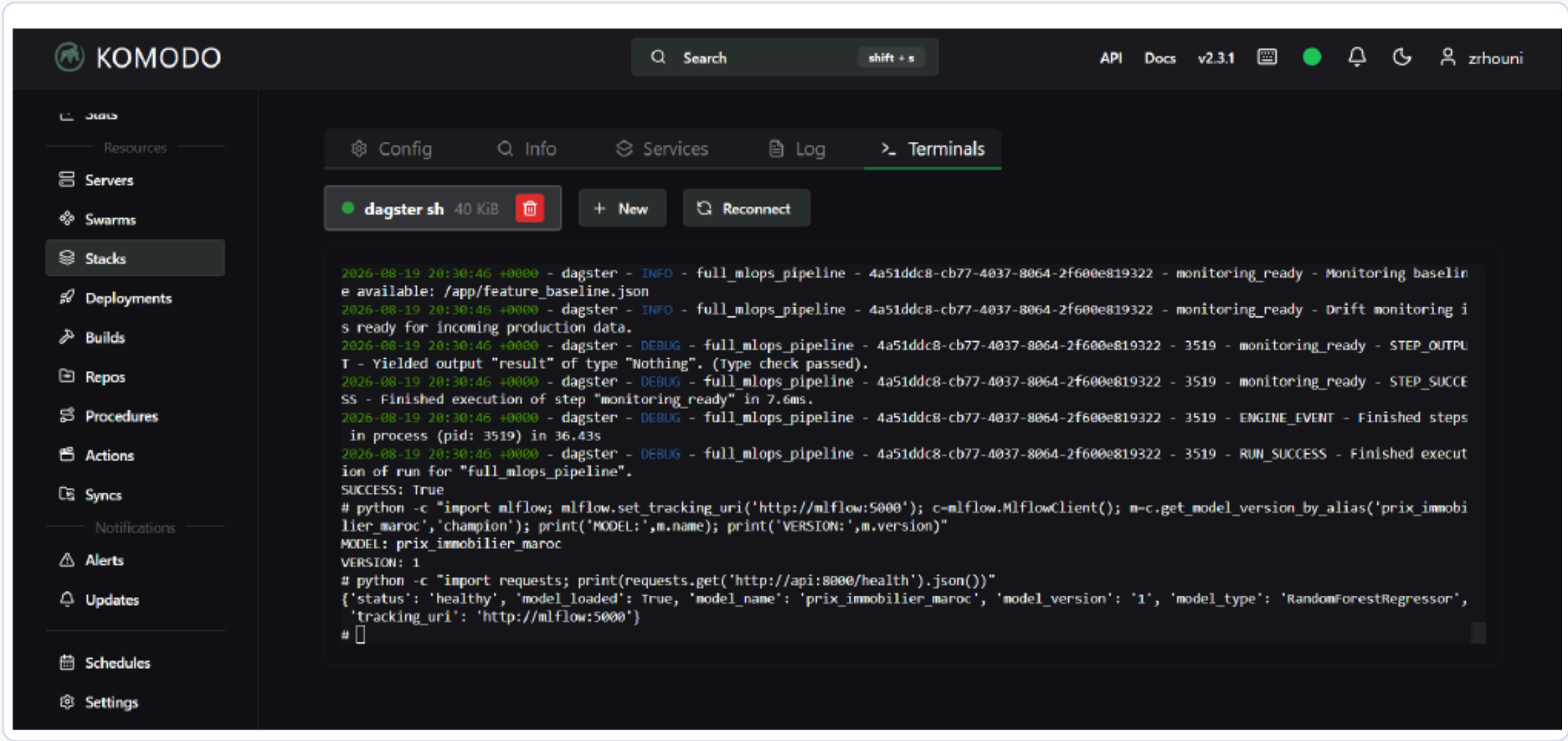
Quality gate

L'entraînement ne démarre qu'après validation des tests dbt.

L'entraînement ne démarre qu'après validation des tests dbt.

Exécution complète du pipeline sur le serveur

Déploiement de la stack sur Komodo



Services opérationnels sur Komodo

Branche

feature/devops-deployment

Réseau

Communication validée entre mlflow:5000, dagster:3000 et api:8000/health

Déploiement

Après Update Stack, un Deploy Stack explicite démarre réellement les services.

Ports hôte utilisés : 3900 (API) · 3901 (MLflow) · 3902 (Dagster) · 3903 (Monitoring)

Monitoring persistant et validation d'une prédiction

HTTP 200

Prédiction valide

≈ 1 420 626

MAD prédicts

3

Endpoints monitoring

20

Échantillons minimum



```
{
  "predicted_price_mad": 1420626.65,
  "model_name": "prix immobilier maroc",
  "model_version": "2",
  "model_type": "RandomForestRegressor",
  "city": "Casablanca",
  "localisation": "Ain Diab",
  "localisation_grouped": "Ain Diab"
}
# not found
# [[A
# python -c "import requests, json; print(json.dumps(requests.get('http://monitoring:8001/drift').json(), indent=2))"
{
  "status": "insufficient_data",
  "samples_analyzed": 1,
  "minimum_required": 20,
  "drift_detected": false,
  "features": {}
}
# ]
```

Figure 17.2: Monitoring de la dérive (drift) avec garde-fou

Validation du endpoint /drift avec garde-fou

Persistence

Les observations sont enregistrées dans availability.jsonl et le service conserve son état via monitoring_data.

Après redémarrage

Le redémarrage explicite de l'API confirme que l'état est rechargé depuis les ressources persistantes.

Problèmes rencontrés et corrections appliquées

Chemins DuckDB incohérents

→ DUCKDB_PATH + configuration par variables d'environnement

API couplée à une version MLflow

→ REGISTERED_MODEL_NAME +
MODEL_ALIAS

Alias champion non géré

→ Affectation explicite de l'alias
champion

État Dagster non partagé

→ Volume dagster_data +
DAGSTER_HOME commun

Baseline drift non persistante

→ Volume monitoring_data

PSI sur trop peu d'observations

→ DRIFT_MIN_SAMPLES = 20

Ces corrections ont permis d'aligner les environnements local, conteneurisé et serveur et de stabiliser toute la chaîne.

Résultats finaux

95 / 95

Story Points

19 / 19

Tests dbt PASS

HTTP 200

API /predict

CI verte

Tests · lint · build

champion

MLflow Registry

RUN SUCCESS

Dagster

Déployé

Komodo

Actif

Monitoring

dlt · DuckDB · dbt · Dagster · scikit-learn · MLflow · FastAPI · Docker Compose · GitHub Actions · Komodo

Collecter → Fiabiliser → Apprendre → Industrialiser